

Proyecto 1 — DNS 2.0

Documentación del proyecto

Tecnológico de Costa Rica

Escuela de Ingeniería en Computación

Curso: Redes IC7602

Proyecto: Proyecto 1 — DNS 2.0

Semestre: I Semestre 2026

Integrantes

- Mariela Solano Gómez
 - Alejandra Delgado Pérez
 - Joshua Obando Castro
 - Roilin Navarro Vargas
 - Esteban Cortes
-

Tabla de contenidos

1. [Introducción](#)
 2. [Arquitectura general del proyecto](#)
 3. [Estructura del repositorio](#)
 4. [DNS API](#)
 5. [DNS Interceptor](#)
 6. [DNS UI](#)
 7. [Estado del Health Checker](#)
 8. [Base de datos](#)
 9. [Ejecución general del proyecto con Docker Compose](#)
 10. [Pruebas realizadas](#)
 11. [Problemas encontrados y soluciones](#)
 12. [Recomendaciones](#)
 13. [Conclusiones](#)
-

Introducción

Este proyecto consiste en implementar un sistema DNS 2.0 utilizando servicios de capa de aplicación sobre protocolos de transporte TCP y UDP. La idea principal es construir un sistema capaz de recibir consultas DNS reales, revisar si el dominio consultado existe dentro de una base de datos propia y responder con una IP configurada por el sistema. Si el dominio no existe en la base de datos, la consulta se reenvía a un DNS público por medio del DNS API.

El sistema está dividido en varios componentes. El DNS Interceptor recibe las consultas DNS por UDP, analiza los paquetes siguiendo la estructura de DNS y decide qué flujo aplicar. El DNS API consulta la base de datos y también permite reenviar paquetes DNS hacia un servidor DNS remoto. La DNS UI permite administrar visualmente los registros DNS desde una aplicación web. Además, el proyecto contempla un Health Checker, encargado de revisar el estado de los registros y actualizar su disponibilidad automáticamente.

La solución se ejecuta usando Docker Compose, lo cual permite levantar los servicios principales con un solo comando. Esto facilita la automatización del proyecto y evita tener que ejecutar cada componente manualmente.

Arquitectura general del proyecto

El sistema funciona con la siguiente arquitectura general:

```
Cliente DNS
  |
  | Consulta DNS por UDP
  v
DNS Interceptor
  |
  | Consulta HTTP
  v
DNS API
  |
  | Consulta datos
  v
Supabase
```

Cuando el dominio sí existe en la base de datos, el flujo esperado es:

```
Cliente DNS
  |
  | Consulta DNS
  v
DNS Interceptor
  |
  | Extrae dominio
  v
DNS API
  |
  | Devuelve IP final
  v
DNS Interceptor
  |
  | Construye respuesta DNS
```

v
Cliente DNS

Cuando el dominio no existe en la base de datos, el flujo cambia:

```
Cliente DNS
|
| Consulta DNS
v
DNS Interceptor
|
| Paquete DNS original codificado en Base64
v
DNS API
|
| Consulta DNS por UDP
v
DNS público
|
| Respuesta DNS
v
DNS API
|
| Respuesta codificada en Base64
v
DNS Interceptor
|
| Respuesta DNS al cliente
v
Cliente DNS
```

En resumen, el flujo general del proyecto es el siguiente:

1. Un cliente realiza una consulta DNS usando `nslookup`.
2. El DNS Interceptor recibe el paquete DNS en UDP/53.
3. El Interceptor analiza el header DNS y extrae el dominio consultado.
4. El Interceptor consulta al DNS API para saber si el dominio existe.
5. Si el dominio existe y está saludable, el Interceptor construye una respuesta DNS válida.
6. Si el dominio no existe o está marcado como no saludable, el Interceptor reenvía el paquete original al DNS API en Base64.
7. El DNS API consulta un DNS remoto y devuelve la respuesta al Interceptor.
8. El Interceptor devuelve la respuesta final al cliente DNS.

Estructura del repositorio

La estructura general del proyecto es la siguiente:

```
P1/
├── dns-api/
│   ├── dns/
│   ├── supabase/
│   ├── Dockerfile
│   ├── docker-compose.yml
│   ├── go.mod
│   ├── go.sum
│   ├── main.go
│   ├── .env
│   └── README.md
├── dns-interceptor/
│   ├── src/
│   │   ├── api_client.cpp
│   │   ├── api_client.h
│   │   ├── base64.cpp
│   │   ├── base64.h
│   │   ├── dns_parser.cpp
│   │   ├── dns_parser.h
│   │   ├── dns_response.cpp
│   │   ├── dns_response.h
│   │   ├── main.cpp
│   │   ├── query_handler.cpp
│   │   ├── query_handler.h
│   │   ├── udp_server.cpp
│   │   └── udp_server.h
│   ├── CMakeLists.txt
│   ├── Dockerfile
│   └── .dockerignore
├── dns-ui/
│   ├── src/
│   ├── Dockerfile
│   ├── index.html
│   ├── nginx.conf
│   ├── package.json
│   ├── vite.config.js
│   └── README.md
└── docker-compose.yml
```

DNS API

Descripción

El DNS API es una API REST implementada en Go. Su responsabilidad principal es servir como punto intermedio entre el DNS Interceptor y la base de datos. Este componente consulta los registros almacenados

en Supabase y devuelve la información necesaria para que el Interceptor pueda responder correctamente una consulta DNS.

También tiene un endpoint que permite recibir paquetes DNS codificados en Base64, decodificarlos, enviarlos a un servidor DNS remoto y devolver la respuesta nuevamente codificada en Base64.

En este proyecto, el DNS API es importante porque evita que el Interceptor tenga que conectarse directamente a la base de datos. Esto ayuda a separar responsabilidades: el Interceptor se encarga de recibir y responder DNS, mientras que el API se encarga de consultar datos y aplicar lógica de resolución.

Responsabilidades principales

El DNS API tiene las siguientes responsabilidades:

- Consultar si un dominio existe en la base de datos.
- Devolver la IP final que debe usar el DNS Interceptor.
- Manejar registros de tipo `single`, `multi`, `weight`, `geo` y `round-trip`.
- Reenviar consultas DNS externas a un DNS público cuando el dominio no existe localmente.
- Comunicarse con Supabase.
- Recibir y devolver paquetes DNS codificados en Base64.

Endpoints principales

GET /api/exists

Este endpoint permite verificar si un dominio existe en la base de datos y obtener la IP que debe responder el DNS Interceptor.

Ejemplo:

```
curl.exe "http://localhost:8080/api/exists?domain=single-test.com&client_ip=127.0.0.1"
```

Respuesta que se espera:

```
{
  "exists": true,
  "healthy": true,
  "type": "single",
  "ip": "192.168.1.50",
  "ttl": 300
}
```

Los campos más importantes son:

Campo	Descripción
<code>exists</code>	Indica si el dominio existe en la base de datos.
<code>healthy</code>	Indica si el registro está saludable.
<code>type</code>	Tipo de registro: <code>single</code> , <code>multi</code> , <code>weight</code> , <code>geo</code> o <code>round-trip</code> .
<code>ip</code>	IP final que debe responder el DNS Interceptor.
<code>ttd</code>	Tiempo de vida del registro DNS.

POST /api/dns_resolver

Este endpoint recibe un paquete DNS codificado en Base64. Luego lo decodifica, lo envía a un DNS remoto y devuelve la respuesta también en Base64.

Este endpoint se usa cuando:

- El dominio no existe en la base de datos.
- El dominio existe, pero está marcado como `unhealthy`.
- El paquete DNS recibido no es una query estándar.

Flujo:

```
Paquete DNS original
  ↓
Base64
  ↓
POST /api/dns_resolver
  ↓
DNS API decodifica
  ↓
Consulta DNS remoto
  ↓
Respuesta DNS
  ↓
Base64
  ↓
Respuesta al DNS Interceptor
```

Tipos de registros soportados

Registro `single`

Este tipo de registro devuelve una única IP.

Ejemplo en Supabase:

```
[
  {
```

```
"ip": "192.168.1.50",  
"weight": 1,  
"country": ""  
}  
]
```

Resultado esperado:

```
{  
  "exists": true,  
  "healthy": true,  
  "type": "single",  
  "ip": "192.168.1.50",  
  "ttl": 300  
}
```

Registro **multi**

Este tipo de registro contiene varias IPs. La idea es que diferentes consultas puedan devolver diferentes IPs, simulando una estrategia de round-robin.

Ejemplo:

```
[  
  {  
    "ip": "10.10.10.1"  
  },  
  {  
    "ip": "10.10.10.2"  
  },  
  {  
    "ip": "10.10.10.3"  
  }  
]
```

Registro **weight**

Este tipo de registro permite asignar pesos a las IPs. La IP con mayor peso debería atender más consultas que las demás.

Ejemplo:

```
[  
  {  
    "ip": "11.11.11.11",  
    "weight": 5  
  },  
]
```

```
{
  "ip": "22.22.22.22",
  "weight": 1
}
```

Registro **geo**

Este tipo de registro selecciona una IP dependiendo del país de origen del cliente. Para esto se usa la IP del cliente y la tabla **ip_to_country**.

Ejemplo:

```
[
  {
    "ip": "10.0.0.1",
    "country": "CR"
  },
  {
    "ip": "10.0.0.2",
    "country": "US"
  }
]
```

Registro **round-trip**

Este tipo de registro selecciona la IP con menor latencia disponible. Para esto, el DNS API utiliza la información generada por el Health Checker en la tabla **health_results**.

El flujo es el siguiente:

```
DNS API recibe consulta por un dominio round-trip
↓
Busca el registro en dns_records
↓
Consulta los targets asociados al registro
↓
Lee los últimos resultados de health_results
↓
Filtra los targets saludables
↓
Selecciona la IP con menor latency_ms
↓
Devuelve esa IP al DNS Interceptor
```

La latencia se mide desde una ubicación simulada del Health Checker. En nuestro caso, se configuró una ubicación en Cartago, Costa Rica:


```
checker_location_id = CR-01
checker_country = CR
checker_city = Cartago
```

Esto permite simular que las mediciones de round-trip time se hacen desde una región específica. Para las pruebas, se usaron resultados reales del Health Checker y también se puede simular latencias específicas para comprobar que el DNS API devuelve la IP con menor `latency_ms`.

Ejemplo:

```
8.8.8.8 -> 80 ms
8.8.4.4 -> 20 ms
```

Entonces para un registro `round-trip`, el API debería devolver:

```
{
  "exists": true,
  "healthy": true,
  "type": "round-trip",
  "ip": "8.8.4.4",
  "ttl": 300
}
```

Variables de entorno

El DNS API utiliza un archivo `.env` dentro de la carpeta `dns-api`.

Ejemplo:

```
SUPABASE_URL=https://xxxxxxx.supabase.co
SUPABASE_KEY=xxxxxxxxxxxxxxxxxxxxxx
REMOTE_DNS_SERVER=8.8.8.8
PORT=8080
```

Por seguridad, el archivo `.env` real no debería subirse al repositorio público. Lo recomendable es tener un archivo `.env.example` con la estructura, pero sin credenciales reales.

Dockerfile del DNS API

El DNS API se ejecuta dentro de un contenedor Docker. Su Dockerfile debe construir la aplicación Go y exponer el puerto 8080.

Ejemplo general:

```
FROM golang:1.23-alpine AS builder

WORKDIR /app

COPY go.mod go.sum ./
RUN go mod download

COPY . .

RUN go build -o dns-api .

FROM alpine:latest

WORKDIR /app

COPY --from=builder /app/dns-api .

EXPOSE 8080

CMD ["./dns-api"]
```

Prueba individual del DNS API

Para probar el API directamente:

```
curl.exe "http://localhost:8080/api/exists?domain=single-
test.com&client_ip=127.0.0.1"
```

Resultado observado durante las pruebas:

```
{
  "country": "",
  "ip": "192.168.1.50",
  "weight": 1,
  "healthy": true
}
```

Luego se ajustó el flujo para que el API pueda devolver una respuesta más útil para el Interceptor, con campos como `exists`, `healthy`, `type`, `ip` y `ttn`.

La estructura que se usó para el Interceptor es:

```
{
  "exists": true,
  "healthy": true,
  "type": "single",
```

```
"ip": "192.168.1.50",  
"ttl": 300  
}
```

DNS Interceptor

Descripción

El DNS Interceptor es un servidor DNS implementado en C++. Este componente escucha consultas DNS en UDP/53. Su trabajo es recibir paquetes DNS reales, revisar si son consultas estándar, extraer el dominio consultado y decidir qué hacer con la solicitud.

Responsabilidades principales

El DNS Interceptor se encarga de:

- Escuchar consultas DNS por UDP.
- Leer los bytes del paquete DNS.
- Interpretar el header DNS.
- Revisar si el paquete es una query estándar.
- Extraer el dominio consultado.
- Consultar al DNS API.
- Construir una respuesta DNS válida si el dominio existe.
- Reenviar la consulta al DNS API si el dominio no existe.
- Devolver la respuesta final al cliente DNS.

Flujo general del Interceptor

```
Cliente DNS  
  |  
  | Consulta DNS  
  v  
DNS Interceptor  
  |  
  | Lee header DNS  
  | Extrae dominio  
  v  
DNS API  
  |  
  | Devuelve IP o indica fallback  
  v  
DNS Interceptor  
  |  
  | Responde al cliente  
  v  
Cliente DNS
```

Flujo 1: query estándar

Una query estándar es aquella donde:

```
QR = 0
OPCODE = 0
```

Esto significa que el paquete recibido es una consulta DNS normal.

Flujo:

1. El cliente pregunta por un dominio.
2. El Interceptor recibe el paquete DNS.
3. El Interceptor lee el header.
4. El Interceptor valida QR = 0 y OPCODE = 0.
5. El Interceptor extrae el dominio.
6. El Interceptor consulta al DNS API.
7. Si el dominio existe y está healthy, construye una respuesta DNS.
8. Si no existe o está unhealthy, reenvía el paquete al DNS API en Base64.

Flujo 2: paquete no estándar

Si el paquete recibido no es una query estándar, el Interceptor no intenta resolverlo localmente.

Flujo:

1. El Interceptor recibe un paquete DNS.
2. Detecta que no es query estándar.
3. Codifica el paquete completo en Base64.
4. Hace POST a /api/dns_resolver.
5. Recibe la respuesta en Base64.
6. Decodifica la respuesta.
7. La devuelve al cliente.

¿Qué es Base64 en este proyecto?

DNS trabaja con paquetes binarios. Es decir, la consulta no llega como texto normal, sino como bytes.

Ejemplo:

```
1A 2B 01 00 00 01 00 00 00 00 00 00
03 77 77 77 03 74 65 63 02 61 63 02 63 72 00
00 01 00 01
```

Para enviar ese paquete por HTTP al DNS API, se convierte a Base64. Esto no es cifrado ni compresión. Solo es una forma de representar bytes como texto.

Ejemplo:

```
Bytes DNS → Base64 → HTTP → Base64 → Bytes DNS
```

En el caso de fallback:

```
Paquete DNS original
↓
Base64
↓
POST /api/dns_resolver
↓
Respuesta Base64
↓
Bytes DNS
↓
Respuesta al cliente
```

Archivos principales del Interceptor

Archivo	Descripción
<code>main.cpp</code>	Por donde ingresamos al interceptor. Lee el puerto desde variable de entorno y arranca el servidor.
<code>udp_server.cpp</code> / <code>udp_server.h</code>	Implementa el servidor UDP que recibe consultas DNS.
<code>dns_parser.cpp</code> / <code>dns_parser.h</code>	Lee el header DNS, obtiene QR, OPCODE y extrae el dominio.
<code>query_handler.cpp</code> / <code>query_handler.h</code>	Implementa la lógica de decisión.
<code>api_client.cpp</code> / <code>api_client.h</code>	Permite comunicarse con el DNS API usando HTTP.
<code>base64.cpp</code> / <code>base64.h</code>	Codifica y decodifica paquetes en Base64.
<code>dns_response.cpp</code> / <code>dns_response.h</code>	Construye respuestas DNS válidas para registros tipo A.

Dockerfile del Interceptor

```
FROM debian:12-slim

RUN apt-get update && apt-get install -y \
    build-essential \
    cmake \
```

```
&& rm -rf /var/lib/apt/lists/*  
  
WORKDIR /app  
  
COPY . .  
  
RUN cmake -S . -B build && cmake --build build  
  
EXPOSE 53/udp  
  
CMD ["/build/dns_interceptor"]
```

Prueba individual del Interceptor

Para ver logs del Interceptor:

```
docker compose logs -f dns-interceptor
```

Durante las pruebas se observó:

```
[DNS Interceptor] Escuchando en UDP/53  
[SERVER] Escuchando en puerto 53  
[STANDARD] ID=7fb1 dominio=single-test.com  
[1B] Consultando DNS API por dominio=single-test.com client_ip=172.20.0.5  
[1B] Registro local encontrado. type=single ip=192.168.1.50 ttl=300
```

Esto nos confirma que el Interceptor recibió una consulta DNS, extrajo el dominio, consultó al API y encontró un registro local.

DNS UI

Descripción

La DNS UI es una aplicación web desarrollada en React. Su objetivo es permitir la administración visual de los registros DNS sin tener que modificar directamente la base de datos.

Desde esta interfaz se pueden crear, visualizar, editar y eliminar registros relacionados con el sistema DNS 2.0.

La UI se ejecuta dentro de un contenedor Docker y se sirve mediante Nginx. En la configuración actual, se expone en el puerto 3000 del host.

Responsabilidades principales

La DNS UI se encarga de:

- Mostrar los registros DNS almacenados.
- Permitir crear registros nuevos.
- Permitir editar registros existentes.
- Permitir eliminar registros.
- Administrar información relacionada con los tipos de registros.
- Servir como interfaz administrativa para el sistema.

Tipos de registros administrados

La interfaz permite trabajar con registros de tipo:

- `single`
- `multi`
- `weight`
- `geo`
- `round-trip`

Cada registro contiene información como:

Campo	Descripción
<code>domain</code>	Nombre del dominio administrado.
<code>type</code>	Tipo del registro DNS.
<code>ips</code>	Lista de IPs asociadas al dominio.
<code>healthy</code>	Estado de salud del registro.

Puerto de acceso

La UI se expone en:

```
http://localhost:3000
```

En Docker Compose:

```
dns-ui:  
  ports:  
    - "3000:80"
```

Esto lo que significa que Nginx sirve la aplicación en el puerto 80 dentro del contenedor, pero desde la computadora local se accede usando el puerto 3000.

Dockerfile de la UI

Una estructura común para la UI es construir la aplicación con Node y luego servir los archivos estáticos con Nginx.

```
FROM node:20-alpine AS builder

WORKDIR /app

COPY package*.json ./
RUN npm install

COPY . .
RUN npm run build

FROM nginx:alpine

COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Prueba individual de la UI

Una vez levantado el proyecto, se abre en el navegador:

```
http://localhost:3000
```

Resultado esperado:

```
La aplicación web carga correctamente y permite visualizar la interfaz
administrativa.
```

Durante las pruebas con Docker Compose, los logs mostraron que el contenedor `dns-ui` se levantó usando Nginx:

```
nginx/1.29.0
start worker processes
```

Esto nos dice que la aplicación está siendo servida correctamente por Nginx dentro del contenedor.

Health Checker

Descripción

El Health Checker es el componente encargado de revisar periódicamente si los servicios asociados a los registros DNS se encuentran disponibles. Este componente fue implementado en C y se ejecuta dentro de un contenedor Docker.

Su función principal es leer los targets configurados en la base de datos, ejecutar pruebas de salud por HTTP o TCP, guardar los resultados en la tabla `health_results` y actualizar el campo `healthy` de la tabla `dns_records`. De esta forma, el estado calculado por el Health Checker afecta directamente el comportamiento del DNS Interceptor.

El flujo general es el siguiente:

```
Health Checker
  ↓
Lee targets desde Supabase
  ↓
Ejecuta checks HTTP/TCP
  ↓
Guarda resultados en health_results
  ↓
Actualiza dns_records.healthy
  ↓
DNS API lee el estado actualizado
  ↓
DNS Interceptor responde localmente o hace fallback
```

Esto permite que el sistema no responda localmente con una IP que está marcada como no saludable. Si un registro queda como `unhealthy`, el DNS Interceptor deja de responder con la IP configurada y aplica el flujo de fallback hacia `/api/dns_resolver`.

Responsabilidades principales

El Health Checker se encarga de:

- Leer targets desde la tabla `targets`.
- Ejecutar pruebas HTTP.
- Ejecutar pruebas TCP.
- Aplicar reintentos configurables.
- Determinar el estado final por mayoría simple.
- Medir la latencia promedio de cada prueba.
- Guardar resultados en la tabla `health_results`.
- Registrar la ubicación simulada del Health Checker.
- Actualizar el campo `healthy` en `dns_records`.
- Direccionar registros `round-trip` mediante mediciones de latencia.

Ubicación simulada del Health Checker

Para simular desde dónde se ejecuta el Health Checker, se configuraron variables de entorno con información geográfica:

```
CHECKER_LOCATION_ID=CR-01
CHECKER_COUNTRY=CR
CHECKER_CITY=Cartago
CHECKER_LATITUDE=9.8644
CHECKER_LONGITUDE=-83.9194
CHECK_INTERVAL_SECONDS=30
```

Esta información permite identificar que las pruebas de latencia fueron realizadas desde una ubicación simulada en Cartago, Costa Rica. Esto es útil para los registros tipo **round-trip**, ya que estos pueden usar la latencia medida desde distintas ubicaciones para escoger la IP más conveniente.

Funcionamiento de los checks

Para cada target, el Health Checker realiza varios intentos. Por ejemplo, si un target tiene **retries = 3**, el sistema ejecuta tres pruebas y luego determina el resultado final.

Ejemplo:

```
3/3 intentos exitosos -> HEALTHY
2/3 intentos exitosos -> HEALTHY
1/3 intentos exitosos -> UNHEALTHY
0/3 intentos exitosos -> UNHEALTHY
```

Durante las pruebas se observaron logs como los siguientes:

```
[HEALTH_CHECKER] [HTTP] intento 1/3 target=example.com:80 -> UP (68.96ms)
[HEALTH_CHECKER] [HTTP] intento 2/3 target=example.com:80 -> UP (74.20ms)
[HEALTH_CHECKER] [HTTP] intento 3/3 target=example.com:80 -> UP (66.79ms)
[HEALTH_CHECKER] Resultado final target=example.com:80 successes=3/3 -> HEALTHY
(avg 69.98ms)
```

También probó con los casos unhealthy:

```
[HEALTH_CHECKER] [HTTP] intento 1/3 target=test-http-ok:81 -> DOWN (1.58ms)
[HEALTH_CHECKER] [HTTP] intento 2/3 target=test-http-ok:81 -> DOWN (1.64ms)
[HEALTH_CHECKER] [HTTP] intento 3/3 target=test-http-ok:81 -> DOWN (1.62ms)
[HEALTH_CHECKER] Resultado final target=test-http-ok:81 successes=0/3 -> UNHEALTHY
(avg 1.61ms)
```

Integración con DNS

La integración se validó con el dominio de prueba **hc-example.com**.

Primero, cuando el registro estaba marcado como saludable, el DNS Interceptor respondió localmente con la IP configurada:

```
Name:    hc-example.com
Address: 9.9.9.9
```

Luego, al modificar el target para que el Health Checker lo marcara como no saludable, el campo `healthy` de `dns_records` cambió a `false`. Después de eso, al consultar nuevamente el dominio, el Interceptor ya no respondió con `9.9.9.9`, sino que aplicó el flujo de fallback. Como `hc-example.com` no existe en DNS público, la respuesta final fue `NXDOMAIN`.

Esto confirma que el Health Checker no solo guarda resultados, sino que también afecta directamente la resolución DNS del sistema.

Base de datos

Descripción

La base de datos utilizada en el proyecto es Supabase. En ella se almacenan los registros DNS, la información de IP to Country, resultados de health checks y targets asociados.

La tabla principal para las pruebas actuales es:

```
dns_records
```

Tabla `dns_records`

La tabla `dns_records` contiene los registros DNS administrados por el sistema.

Campos principales:

Campo	Tipo	Descripción
<code>id</code>	int8	Identificador del registro.
<code>created_at</code>	timestamptz	Fecha de creación.
<code>domain</code>	text	Dominio configurado.
<code>type</code>	text	Tipo del registro.
<code>ips</code>	jsonb	Lista de IPs y datos asociados.
<code>healthy</code>	bool	Estado del registro.

Ejemplo de registros utilizados:

Dominio	Tipo	Estado
single-test.com	single	true
multi-test.com	multi	true
weight-test.com	weight	true
roundtrip-test.com	round-trip	true
geo-test.com	geo	true
verify-single.com	single	true
verify-multi.com	multi	true
verify-weight.com	weight	true

Registros de prueba que pueden ser usados

Para probar el proyecto sin depender todavía del Health Checker, se pueden usar registros marcados manualmente como saludables.

```
DELETE FROM dns_records
WHERE domain IN (
  'verify-single.com',
  'verify-multi.com',
  'verify-weight.com',
  'verify-unhealthy.com'
);

INSERT INTO dns_records (domain, type, ips, healthy)
VALUES
(
  'verify-single.com',
  'single',
  '[{"ip":"5.55.55.55"}]::jsonb',
  true
),
(
  'verify-multi.com',
  'multi',
  '[{"ip":"10.10.10.1"}, {"ip":"10.10.10.2"}, {"ip":"10.10.10.3"}]::jsonb',
  true
),
(
  'verify-weight.com',
  'weight',
  '[{"ip":"11.11.11.11", "weight":5}, {"ip":"22.22.22.22", "weight":1}]::jsonb',
  true
),
(
  'verify-unhealthy.com',
  'single',
```

```
'[{"ip":"9.9.9.9"}]':jsonb,  
false  
);
```

Ejecución general del proyecto con Docker Compose

Requisitos previos

Para ejecutar el proyecto se necesita:

Para ejecución

- Docker Desktop instalado.
- Docker Compose disponible.
- Archivo `.env` configurado en `dns-api`.
- Acceso a internet.
- Acceso a la base de datos Supabase.
- PowerShell o una terminal equivalente.

Para la API

- Go 1.26+](<https://go.dev/dl/>)
- [Docker](#)
- [Postman](#) (para pruebas manuales)
- Acceso al proyecto en Supabase (credenciales proporcionadas por el equipo)

Para la UI

- React 18 + Vite
- JavaScript (ES6+)
- CSS puro
- nginx (para producción en Docker)

Archivo `docker-compose.yml`

El archivo `docker-compose.yml` se encuentra en la raíz del proyecto, dentro de la carpeta `P1`. Hicimos este archivo para automatizar el proceso y que los contenedores necesarios se levantaran con más facilidad.

```
services:  
  dns-api:  
    build:  
      context: ./dns-api  
      container_name: dns-api  
      env_file:  
        - ./dns-api/.env  
    ports:  
      - "8080:8080"
```

```
restart: unless-stopped

dns-interceptor:
  build:
    context: ./dns-interceptor
  container_name: dns-interceptor
  environment:
    DNS_API_URL: http://dns-api:8080
    DNS_PORT: 53
  ports:
    - "8053:53/udp"
  depends_on:
    - dns-api
  restart: unless-stopped

dns-ui:
  build:
    context: ./dns-ui
  container_name: dns-ui
  ports:
    - "3000:80"
  depends_on:
    - dns-api
  restart: unless-stopped

health-checker:
  build:
    context: ./health-checker
  container_name: health-checker
  env_file:
    - ./health-checker/.env
  depends_on:
    - dns-api
    - test-http-ok
  restart: unless-stopped
```

Levantar el proyecto

Desde la carpeta raíz del proyecto:

```
cd C:\TuRuta\TuRuta\TuRuta\2026-01-2022437963-IC7602\proyectos\P1
```

Ejecutar:

```
docker compose up --build
```

Este comando construye las imágenes y levanta los tres servicios principales:

- `dns-api`
- `dns-interceptor`
- `dns-ui`

Si se quiere ejecutar en segundo plano:

```
docker compose up --build -d
```

Verificar servicios activos

```
docker compose ps
```

Salida que se espera:

NAME	IMAGE	SERVICE	STATUS
dns-api	p1-dns-api	dns-api	Up
dns-interceptor	p1-dns-interceptor	dns-interceptor	Up
dns-ui	p1-dns-ui	dns-ui	Up

Puertos esperados:

dns-api	0.0.0.0:8080->8080/tcp
dns-interceptor	0.0.0.0:8053->53/udp
dns-ui	0.0.0.0:3000->80/tcp

Acceso a cada servicio

Servicio	URL o puerto
DNS API	<code>http://localhost:8080</code>
DNS UI	<code>http://localhost:3000</code>
DNS Interceptor	<code>dns-interceptor:53</code> dentro de la red Docker
DNS Interceptor desde host	<code>127.0.0.1:8053</code>

Ver logs

Todos los servicios:

```
docker compose logs -f
```

Solo DNS API:

```
docker compose logs -f dns-api
```

Solo DNS Interceptor:

```
docker compose logs -f dns-interceptor
```

Solo DNS UI:

```
docker compose logs -f dns-ui
```

Apagar el proyecto

```
docker compose down
```

Pruebas realizadas

Prueba 1 — Verificar que los contenedores están corriendo

Comando:

```
docker compose ps
```

Resultado esperado:

```
dns-api           Up
dns-interceptor   Up
dns-ui            Up
```

Resultado obtenido:

```
dns-api           Up
dns-interceptor   Up
dns-ui            Up
```


Conclusión:

Los tres servicios principales del proyecto se levantaron correctamente con Docker Compose.

Prueba 2 — Verificar DNS API

Comando:

```
curl.exe "http://localhost:8080/api/exists?domain=single-test.com&client_ip=127.0.0.1"
```

Resultado esperado:

```
{
  "exists": true,
  "healthy": true,
  "type": "single",
  "ip": "192.168.1.50",
  "ttl": 300
}
```

Resultado observado:

```
{
  "country": "",
  "ip": "192.168.1.50",
  "weight": 1,
  "healthy": true
}
```

Conclusión:

El API logra consultar la base de datos y obtener información del registro. Se recomienda mantener la respuesta final en un formato uniforme para facilitar el trabajo del Interceptor.

Prueba 3 — Verificar logs del DNS Interceptor

Comando:

```
docker compose logs -f dns-interceptor
```

Resultado observado:

```
[DNS Interceptor] Escuchando en UDP/53  
[SERVER] Escuchando en puerto 53
```

Conclusión:

El DNS Interceptor se levantó correctamente dentro del contenedor y está escuchando internamente en UDP/53.

Prueba 4 — Consulta DNS desde la red Docker

Comando:

```
docker run --rm --network p1_default busybox nslookup single-test.com dns-interceptor
```

Resultado observado:

```
Server:      dns-interceptor  
Address:     172.20.0.3:53  
  
Non-authoritative answer:  
Name:   single-test.com  
Address: 192.168.1.50
```

Logs observados en el Interceptor:

```
[STANDARD] ID=7fb1 dominio=single-test.com  
[1B] Consultando DNS API por dominio=single-test.com client_ip=172.20.0.5  
[1B] Registro local encontrado. type=single ip=192.168.1.50 ttl=300
```

Conclusión:

La prueba fue exitosa. Un cliente DNS dentro de la red Docker pudo consultar al DNS Interceptor y recibir una respuesta DNS válida.

Prueba 5 — Registro tipo **single**

Comando:

```
docker run --rm --network p1_default busybox nslookup single-test.com dns-interceptor
```

Resultado esperado:

```
Name: single-test.com
Address: 192.168.1.50
```

Resultado obtenido:

```
Name: single-test.com
Address: 192.168.1.50
```

Conclusión:

El registro tipo single funciona correctamente, ya que devuelve una única IP configurada en la base de datos.

Prueba 6 — Registro tipo **multi**

Comandos:

```
docker run --rm --network p1_default busybox nslookup multi-test.com dns-interceptor
docker run --rm --network p1_default busybox nslookup multi-test.com dns-interceptor
docker run --rm --network p1_default busybox nslookup multi-test.com dns-interceptor
```

Resultado esperado:

El sistema debe alternar entre las IPs disponibles para el dominio.

Conclusión:

Esta prueba permite validar el comportamiento round-robin de los registros tipo multi.

Prueba 7 — Registro tipo **weight**

Comandos:

```
docker run --rm --network p1_default busybox nslookup weight-test.com dns-interceptor
docker run --rm --network p1_default busybox nslookup weight-test.com dns-interceptor
docker run --rm --network p1_default busybox nslookup weight-test.com dns-interceptor
```

Resultado esperado:

La IP con mayor peso debe aparecer con más frecuencia que las demás.

Conclusión:

Esta prueba permite validar que el API seleccione direcciones considerando el peso configurado en la base de datos.

Prueba 8 — Dominio no existente

Comando:

```
docker run --rm --network p1_default busybox nslookup google.com dns-interceptor
```

Resultado esperado:

El Interceptor debe detectar que el dominio no existe en la base de datos y hacer fallback hacia /api/dns_resolver.

Flujo esperado:

```
Consulta DNS
↓
DNS Interceptor
```

```
↓  
/api/exists  
↓  
No existe  
↓  
Base64  
↓  
/api/dns_resolver  
↓  
DNS remoto  
↓  
Respuesta al cliente
```

Conclusión:

Esta prueba permite validar el flujo de fallback para dominios que no se encuentran en la base de datos.

Prueba 9 — Acceso a la DNS UI

URL:

```
http://localhost:3000
```

Resultado esperado:

La interfaz web carga correctamente.

Conclusión:

La DNS UI queda disponible desde el navegador mediante el puerto 3000.

Prueba 10 — Prueba opcional desde el host

Comando:

```
nslookup -port=8053 single-test.com 127.0.0.1
```

Resultado esperado:

El host debería consultar al Interceptor usando el puerto mapeado 8053.

Observación:

En algunos entornos con Docker Desktop sobre Windows, el tráfico UDP desde el host hacia el contenedor puede comportarse de forma diferente.

Por esa razón, la prueba principal recomendada es la prueba desde un contenedor cliente dentro de la misma red Docker.

Conclusión:

La prueba interna con BusyBox es válida porque realiza una consulta DNS real contra el servicio dns-interceptor en el puerto 53 dentro de la red Docker.

Prueba 11 — Verificar Health Checker en ejecución

Comando:

```
docker compose logs -f health-checker
```

Resultado observado:

```
Health Checker iniciado (Proyecto1_IC7602)...  
[HEALTH_CHECKER] location=CR-01 country=CR city=Cartago lat=9.8644 lon=-83.9194
```

Conclusión:

El Health Checker se levantó correctamente dentro del contenedor y cargó la ubicación simulada configurada mediante variables de entorno.

Prueba 12 — Health Check HTTP saludable

Para esta prueba se usó el target `example.com:80` y también el servicio local de prueba `test-http-ok:80`.

Resultado observado:

```
[HEALTH_CHECKER] [HTTP] intento 1/3 target=example.com:80 -> UP (68.96ms)  
[HEALTH_CHECKER] [HTTP] intento 2/3 target=example.com:80 -> UP (74.20ms)
```

```
[HEALTH_CHECKER] [HTTP] intento 3/3 target=example.com:80 -> UP (66.79ms)
[HEALTH_CHECKER] Resultado final target=example.com:80 successes=3/3 -> HEALTHY
(avg 69.98ms)
```

También se observó:

```
[HEALTH_CHECKER] [HTTP] intento 1/3 target=test-http-ok:80 -> UP (2.69ms)
[HEALTH_CHECKER] [HTTP] intento 2/3 target=test-http-ok:80 -> UP (2.43ms)
[HEALTH_CHECKER] [HTTP] intento 3/3 target=test-http-ok:80 -> UP (2.76ms)
[HEALTH_CHECKER] Resultado final target=test-http-ok:80 successes=3/3 -> HEALTHY
(avg 2.63ms)
```

Conclusión:

La prueba fue exitosa. El Health Checker pudo realizar checks HTTP, medir latencia, aplicar tres intentos y determinar el resultado final como HEALTHY.

Prueba 13 — Health Check HTTP no saludable

Para esta prueba se usó el servicio `test-http-ok`, pero con un puerto incorrecto: `81`.

Resultado observado:

```
[HEALTH_CHECKER] [HTTP] intento 1/3 target=test-http-ok:81 -> DOWN (1.58ms)
[HEALTH_CHECKER] [HTTP] intento 2/3 target=test-http-ok:81 -> DOWN (1.64ms)
[HEALTH_CHECKER] [HTTP] intento 3/3 target=test-http-ok:81 -> DOWN (1.62ms)
[HEALTH_CHECKER] Resultado final target=test-http-ok:81 successes=0/3 -> UNHEALTHY
(avg 1.61ms)
```

Conclusión:

El Health Checker detectó correctamente un servicio HTTP no disponible y lo marcó como UNHEALTHY por mayoría simple.

Prueba 14 — Health Check TCP saludable

Para esta prueba se usó el servidor DNS público de Google en el puerto 53.

Resultado observado:

```
[HEALTH_CHECKER] [TCP] intento 1/3 target=8.8.8.8:53 -> UP (76.63ms)
[HEALTH_CHECKER] [TCP] intento 2/3 target=8.8.8.8:53 -> UP (87.16ms)
[HEALTH_CHECKER] [TCP] intento 3/3 target=8.8.8.8:53 -> UP (101.65ms)
[HEALTH_CHECKER] Resultado final target=8.8.8.8:53 successes=3/3 -> HEALTHY (avg 88.48ms)
```

Conclusión:

La prueba TCP fue exitosa. El Health Checker pudo abrir conexión contra 8.8.8.8 en el puerto 53, medir latencia y marcar el target como HEALTHY.

Prueba 15 — Health Check TCP no saludable

Para esta prueba se usó el servicio `test-http-ok`, pero con un puerto cerrado: `9999`.

Resultado observado:

```
[HEALTH_CHECKER] [TCP] intento 1/3 target=test-http-ok:9999 -> DOWN (0.95ms)
[HEALTH_CHECKER] [TCP] intento 2/3 target=test-http-ok:9999 -> DOWN (1.05ms)
[HEALTH_CHECKER] [TCP] intento 3/3 target=test-http-ok:9999 -> DOWN (0.91ms)
[HEALTH_CHECKER] Resultado final target=test-http-ok:9999 successes=0/3 -> UNHEALTHY (avg 0.97ms)
```

Conclusión:

El Health Checker detectó correctamente un puerto TCP cerrado y marcó el target como UNHEALTHY.

Prueba 16 — Verificar resultados guardados en Supabase

Consulta usada:

```
SELECT
  hr.id,
  t.domain_name,
  t.ip_address,
  t.port,
  t.check_type,
  hr.is_healthy,
  hr.latency_ms,
  hr.checker_location_id,
  hr.checked_at
```



```
FROM health_results hr
JOIN targets t ON t.id = hr.target_id
ORDER BY hr.checked_at DESC
LIMIT 20;
```

Resultado esperado:

La tabla health_results debe mostrar resultados recientes con:

- is_healthy = true o false
- latency_ms con la latencia medida
- checker_location_id = CR-01
- checked_at con la fecha y hora del check

Conclusión:

Se confirmó que el Health Checker no solo imprime resultados en logs, sino que también guarda el historial de revisiones en la base de datos.

Prueba 17 — Integración Health Checker con DNS Interceptor

Para esta prueba se creó el dominio `hc-example.com` en `dns_records` con la IP `9.9.9.9`.

Cuando el target asociado estaba saludable, el DNS Interceptor respondió localmente:

```
docker run --rm --network p1_default busybox nslookup hc-example.com dns-
interceptor
```

Resultado observado:

```
Server:      dns-interceptor
Address:     172.20.0.6:53

Non-authoritative answer:
Name:   hc-example.com
Address: 9.9.9.9
```

Luego se modificó el target asociado para que fallara y el Health Checker actualizó el registro como `healthy = false`. Al consultar nuevamente:

```
docker run --rm --network p1_default busybox nslookup hc-example.com dns-interceptor
```

Resultado observado:

```
server can't find hc-example.com: NXDOMAIN
```

Conclusión:

La prueba fue exitosa. Cuando el registro estaba healthy, el Interceptor respondió localmente con 9.9.9.9. Cuando el Health Checker lo marcó como unhealthy, el Interceptor dejó de responder localmente y aplicó fallback. Como hc-example.com no existe en DNS público, el resultado final fue NXDOMAIN.

Prueba 18 — Registro round-trip usando latencia del Health Checker

Para validar el tipo `round-trip`, se creó un registro con varias IPs candidatas y targets asociados. El Health Checker midió la latencia hacia cada target y guardó los resultados en `health_results`.

Flujo validado:

```
DNS API recibe consulta por dominio round-trip
↓
Consulta targets asociados al dns_record_id
↓
Lee últimos resultados en health_results
↓
Filtra targets saludables
↓
Selecciona la IP con menor latency_ms
↓
Devuelve esa IP al DNS Interceptor
```

Consulta de ejemplo al API:

```
curl.exe "http://localhost:8080/api/exists?domain=roundtrip-health.com&client_ip=127.0.0.1"
```

Resultado esperado:

```
{  
  "exists": true,  
  "healthy": true,  
  "type": "round-trip",  
  "ip": "IP_CON_MENOR_LATENCIA",  
  "ttl": 300  
}
```

Conclusión:

La prueba valida que el registro round-trip puede usar la información generada por el Health Checker para seleccionar la IP saludable con menor latencia.

Problemas encontrados y soluciones

Problema 1 — `make` no se reconocía en Windows

Mensaje observado:

```
make: The term 'make' is not recognized
```

Causa:

En Windows con Visual Studio/MSVC no se usa make directamente.

Solución:

Se intentó usar `cmake --build`, pero luego se identificó que el código del Interceptor usa headers propios de Linux.

Problema 2 — Headers de Linux no encontrados en Windows

Errores observados:

```
netinet/in.h: No such file or directory  
arpa/inet.h: No such file or directory  
netdb.h: No such file or directory
```

Causa:

El DNS Interceptor usa sockets POSIX de Linux. Estos headers no existen en Windows/MSVC.

Solución:

Se decidió compilar y ejecutar el Interceptor dentro de un contenedor Debian mediante Docker.

Problema 3 — Consulta desde Windows no llegaba al Interceptor

Mensaje observado:

No response from server

Causa probable:

El tráfico UDP desde Windows hacia Docker Desktop presentaba problemas dependiendo del entorno.

Solución:

Se realizó la prueba desde un contenedor BusyBox dentro de la misma red Docker. Esto permitió validar correctamente el funcionamiento del DNS Interceptor.

Problema 4 — Timeout al consultar el DNS remoto 8.8.8.8

Mensaje observado:

read udp 172.20.0.2:47457->8.8.8.8:53: i/o timeout

Causa:

El DNS Interceptor sí entró correctamente al flujo de fallback hacia /api/dns_resolver. Sin embargo, el DNS API no recibió respuesta del servidor DNS remoto 8.8.8.8 dentro del tiempo configurado. Esto puede deberse a conectividad

UDP desde Docker hacia Internet, firewall, bloqueo de red o una respuesta tardía del DNS remoto.

Solución:

Se verificó que el flujo del Interceptor era correcto, ya que los logs mostraron que detectó el dominio como no existente o unhealthy y llamó al fallback. El problema se ubicó en la comunicación del DNS API con el DNS remoto. Se mantuvo 8.8.8.8 como servidor DNS remoto principal, configurado mediante variable de entorno, tal como se espera para el proyecto.

Recomendaciones

1. Mantener Docker Compose como forma principal de ejecución, ya que permite levantar todos los servicios con un solo comando.
2. Siempre tener el `dns-api` corriendo antes de abrir la UI, de lo contrario la tabla aparecerá vacía.
3. Probar el DNS Interceptor desde la red Docker usando BusyBox, porque esta prueba es más estable y más parecida al entorno real de contenedores.
4. Mantener el archivo `.env` fuera del repositorio público para evitar exponer credenciales de Supabase.
5. El campo `healthy` lo actualiza el `health-checker` automáticamente — no modificarlo manualmente desde la UI.
6. Mantener separados los componentes del proyecto: API, Interceptor, UI y Health Checker.
7. Evitar que el Interceptor se conecte directamente a la base de datos. Es mejor que consulte al DNS API.
8. Hacer pruebas por capas: primero API, luego Interceptor, luego UI y finalmente integración completa.
9. Agregar pruebas unitarias para funciones críticas como parseo DNS, Base64, selección de IP y construcción de respuestas DNS.
10. Probar primero registros simples tipo `single` antes de validar registros más complejos como `geo` o `round-trip`.
11. Usar logs descriptivos para facilitar la revisión del profesor y la depuración del sistema.
12. Documentar los problemas encontrados, porque ayudan a justificar decisiones técnicas tomadas durante el desarrollo.

Conclusiones

1. Implementar un sistema DNS desde cero permitió comprender en profundidad cómo funciona el protocolo DNS a nivel de paquetes, incluyendo la estructura del header, el parseo de queries y la construcción manual de respuestas válidas.
2. Trabajar con múltiples tecnologías en paralelo (Go, C++, React, Docker, Supabase) demostró la importancia de definir interfaces claras entre componentes desde el inicio, ya que cualquier cambio en el API afectaba directamente al Interceptor y a la UI.
3. El uso de Docker Compose simplificó significativamente el proceso de integración, ya que permitió levantar todos los servicios con un solo comando y reproducir el entorno de manera consistente entre los integrantes del equipo.
4. El mapeo `8053:53/udp` es útil para pruebas locales porque evita conflictos con servicios DNS del sistema operativo.
5. El problema del timeout con el DNS remoto evidenció la importancia de separar correctamente los logs por componente, ya que esto permitió identificar con precisión que el flujo del Interceptor era correcto y que el fallo estaba en la comunicación del DNS API con el servidor externo.
6. El timeout al consultar el DNS remoto 8.8.8.8 mostró que la conectividad UDP desde contenedores Docker hacia Internet puede verse afectada por restricciones de red o firewall, lo que debe considerarse al diseñar sistemas que dependan de servicios externos.
7. La separación entre Interceptor y API mejora el diseño, porque cada componente tiene una responsabilidad más clara.
8. La DNS UI facilita la administración de registros porque permite manejar datos desde una interfaz web.
9. La prueba con BusyBox dentro de la red Docker es válida porque usa una consulta DNS real contra el servicio `dns-interceptor`.
10. El Health Checker demostró ser un componente clave para la resiliencia del sistema, ya que al marcar registros como unhealthy el Interceptor puede aplicar fallback automáticamente sin intervención manual.
11. Implementar múltiples tipos de registros (single, multi, weight, geo, round-trip) permitió entender cómo los sistemas DNS reales aplican estrategias de balanceo y distribución geográfica del tráfico.
12. Realizar las pruebas por capas, comenzando por el API de forma aislada y avanzando hasta la integración completa, facilitó la detección temprana de errores y redujo el tiempo necesario para depurar problemas en el sistema.